

VAE - Diffusion

Nguyễn Hoàng Nguyễn

Bản full: [here](#)

1 Variational Autoencoder (VAE)

1.1 Motivation

Một **Autoencoder (AE)** thông thường học cách:

- **Encoder** Nén dữ liệu đầu vào x thành một biểu diễn tiềm ẩn z (latent space).
- **Decoder** Giải nén z để tái tạo lại $\hat{x} \approx x$.

Hạn chế:

- AE chỉ học từ một latent vector z cố định cho mỗi mẫu, nên không thể sinh dữ liệu mới thực sự.
- Latent space z không được ràng buộc theo một phân phối cụ thể, dẫn đến dữ liệu sinh ra có phân phối hỗn loạn (nhiều).

1.2 Ý tưởng của VAE

Variational Autoencoder (VAE) ra đời để khắc phục vấn đề trên.

- Thay vì ánh xạ $x \rightarrow z$ cố định, VAE ánh xạ $x \rightarrow$ một phân phối xác suất $q_\phi(z|x)$, thường là Gaussian $\mathcal{N}(\mu(x), \sigma^2(x))$.
- Encoder sinh ra hai vector: trung bình $\mu(x)$ và độ lệch chuẩn $\sigma(x)$.
- Từ đó, lấy mẫu $z \sim q_\phi(z|x)$, rồi đưa z vào decoder để sinh ra $\hat{x}$.

Nhờ vậy, VAE có thể **sinh dữ liệu mới** — vì nó học phân phối xác suất tiềm ẩn chứ không chỉ mã hoá một điểm cố định.

$$x \rightarrow \text{Encoder} \rightarrow (\mu, \sigma) \rightarrow \text{Sampling } z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

- **Encoder** $q_\phi(z|x)$: mạng nơ-ron dự đoán $\mu(x)$ và $\sigma(x)$.
- **Decoder** $p_\theta(x|z)$: mạng nơ-ron sinh ra $\hat{x}$ từ latent vector z .
- **Latent space** z : không gian đặc trưng ẩn, nơi mỗi điểm tương ứng với một kiểu đặc trưng dữ liệu.

1.3 Reparameterization Trick

Vì không thể lấy đạo hàm trực tiếp qua phép lấy mẫu ngẫu nhiên $z \sim q_\phi(z|x)$, ta sử dụng **reparameterization trick**:

$$z = \mu(x) + \sigma(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Do đó:

$$\mathbb{E}_{z \sim q_\phi(z|x)}[f(z)] = \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)}[f(\mu_\phi(x) + \sigma_\phi(x) \odot \epsilon)]$$

Nhờ đó, gradient chỉ chảy qua μ và σ , còn ϵ là nhiễu cố định từ Gaussian. Điều này cho phép huấn luyện bằng **backpropagation** bình thường.

1.4 VAE Loss

Mục tiêu của VAE là **tối đa hóa** xác suất sinh dữ liệu $p_\theta(x)$:

$$\log p_\theta(x) = \text{ELBO} + \text{KL divergence term } (D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p_\theta(\mathbf{z}|\mathbf{x})))$$

Do $p_\theta(x)$ khó tính trực tiếp, ta tối đa hoá **ELBO (Evidence Lower Bound)**:

$$\begin{aligned} \mathcal{L}(\theta, \phi; x) &= \text{ELBO} \\ &= \mathbb{E}_{z \sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) \parallel p_\theta(z)) \end{aligned}$$

Và cũng là tối thiểu hóa làm mất mát:

$$\begin{aligned} \mathcal{L}_{\text{VAE}}(x) &= -\text{ELBO} \\ &= -\mathbb{E}_{z \sim q_\phi(z|x)}[\log p_\theta(x|z)] + D_{KL}(q_\phi(z|x) \parallel p_\theta(z)) \end{aligned}$$

Tổng hợp lại, hàm mất mát của VAE là:

$$\text{Loss}(x) = \underbrace{D_{KL}(q_\phi(z|x) \parallel p_\theta(z))}_{\text{KL Divergence Loss}} + \underbrace{\text{ReconstructionLoss}(x, \hat{x})}_{\text{Reconstruction Loss}}$$

Trong thực tế, công thức cụ thể thường được viết như sau ($p_\theta(x | z)$ theo phân phối Gauss):

$$\text{Loss}(x) = \frac{1}{2} \sum_i (\mu_i^2 + \sigma_i^2 - \log \sigma_i^2 - 1) + \|x - \hat{x}\|^2$$

2 Denoising Diffusion Probabilistic Models (DDPM)

2.1 Motivation

Diffusion Model là mô hình sinh dữ liệu (generative model) mô phỏng quá trình khuếch tán:

- Forward: từ dữ liệu thật $\rightarrow$ thêm noise dần cho đến khi thành phân phối Gaussian thuần.
- Reverse: học cách khử noise để từ noise $\rightarrow$ tái tạo dữ liệu.

Như vậy:

- Nếu ta biết cách thêm noise, thì ta có thể học cách loại noise để sinh ngược lại dữ liệu.
- Mô hình học phân phối dữ liệu thông qua việc dự đoán noise đã được thêm vào, thay vì dự đoán trực tiếp data (trực tiếp tái tạo data thường khó hơn rất nhiều).

2.2 Forward process

Forward process là quá trình thêm dần nhiễu vào ảnh (hoặc dữ liệu) cho đến khi trở thành noise thuần.

Ta có dữ liệu gốc:

$$x_0 \sim q(x_0)$$

Ở mỗi bước t (từ $1 \rightarrow T$), ta thêm noise theo:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

Trong đó:

- β_t : lượng noise thêm tại bước t (noise schedule), với $\beta_t \in (0, 1)$
- x_t : dữ liệu sau khi đã bị thêm noise t lần
- $\sqrt{1 - \beta_t} x_{t-1}$: mean của phân phối $q(x_t | x_{t-1})$
- $\beta_t I$: variance của phân phối $q(x_t | x_{t-1})$

Reparameterization trick ta có:

$$\begin{aligned} x_t &= \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon \\ &= \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \end{aligned}$$

Suy ra:

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I)$$

Trong đó:

$$\alpha_t = 1 - \beta_t \quad \bar{\alpha}_t = \alpha_t \alpha_{t-1} \dots \alpha_1$$

2.3 Reverse process

Reverse diffusion là quá trình mô phỏng bước chuyển ngược trong mô hình khuếch tán, dựa trên việc ước lượng lại nhiễu đã được thêm ở forward process. Mục đích là mô hình hoá phân phối ngược $p(x_{t-1} | x_t)$ sao cho nếu khởi tạo

$$x_T \sim \mathcal{N}(0, I),$$

rồi thực hiện quá trình sampling ngược dần về x_0 , ta sẽ thu được dữ liệu giống thật.

Tuy nhiên, phân phối đúng $p(x_{t-1} | x_t)$ không tính được trực tiếp vì muốn tính nó phải biết phân phối posterior của x_0 , trong khi x_0 là thứ ta chưa biết khi sampling.

Dựa trên phân phối forward $q(x_t | x_{t-1})$, ta có thể suy ra phân phối xấp xỉ posterior:

$$q(x_{t-1} | x_t, \hat{x}_0),$$

để dùng như một approximation cho $p(x_{t-1} | x_t)$.

Như vậy ta có:

$$q(x_{t-1} | x_t, \hat{x}_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, \hat{x}_0), \tilde{\beta}_t I).$$

Suy ra bước sampling ngược:

$$x_{t-1} = \tilde{\mu}_t(x_t, \hat{x}_0) + \sqrt{\tilde{\beta}_t} z, \quad z \sim \mathcal{N}(0, I).$$

Trong đó:

- $\tilde{\mu}_t(x_t, \hat{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \hat{x}_0 + \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$
- $\tilde{\beta}_t = \frac{\beta_t(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}$
- $\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}}{\sqrt{\bar{\alpha}_t}}$ (Dựa theo công thức forward tính ngược lại x_0)
- $\hat{\epsilon} = \epsilon_\theta(x_t, t)$ (Mô hình học và dự đoán noise này)
- $x_t \sim \mathcal{N}(0, I)$ (với $t=T$)

2.4 DDPM Loss

$$L_{\text{simple}} = \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

Trong đó:

- $\epsilon \sim \mathcal{N}(0, I)$
- $\epsilon_\theta(x_t, t)$ (Mô hình học và dự đoán noise này)